

CivicMysuru — Decision Log

Team Zenith · Team ID HM26-7096 · HackMysuru 1.0, Phase 1 · Sub-problem: Routing

Q1. What approach did we take, and what did we reject?

Approach: the citizen's area + issue type + filing date → look up the jurisdiction record in force on that date → authority, department, priority and verification flags → that authority's staff queue. Every area-to-authority mapping is a dated record (valid_from / valid_to). When a boundary changes, an admin calls POST /api/admin/jurisdiction-change: the open record closes, a new one opens, and the next complaint routes differently with no code change or redeploy. An area with no record, or with overlapping records, is labelled UNASSIGNED or DISPUTED and sent to human review instead of being guessed.

Rejected: GPS point-in-polygon routing against a ward-boundary file. It looked attractive because the citizen never types an area and coordinates cannot be misspelt. We first built a static area dictionary, replaced it, and chose not to build the polygon router.

Q2. Why did we reject it? What was the trade-off?

We compared the two on four dimensions. **Boundary agility:** a versioned record changes with one API call; polygons need officially published shapes for panchayats whose boundaries are still disputed this year. **Offline resilience:** a typed area works with no GPS fix; polygon routing fails indoors or on weak signal. **Build effort in 72 hours:** a table versus sourcing, validating and indexing geometry. **Spam resistance:** polygons win, because GPS can contradict a false location; our typed area cannot.

Cost we knowingly accepted: matching is exact, so a misspelt or over-specific area ("Vijayanagar 2nd Stage", which we tested) returns UNASSIGNED and costs a human review, and a citizen can lie about the area. The jurisdiction table also lives in memory, so an admin change is lost on restart until it moves into SQLite. We preferred an honest "needs review" to a confident wrong route.

Q3. What breaks at the scale of all of Mysuru?

Assume 65 wards, the absorbed bodies, about 5,000 reports a day and Dasara spikes. **The first thing to break is GET /api/complaints:** it returns the whole table with no limit and the dashboard renders every row, while /api/stats also loads every row into Python. At roughly 0.9 KB per complaint (our estimate, not load-tested) the response grows about 4.5 MB a day and passes 25 MB within a week. Next, duplicate_check runs a leading-wildcard ILIKE that scans the full table on every submission, and complaint IDs are read-maximum-plus-one, so two simultaneous submissions can collide.

Fix: keyset pagination (50 rows) with status, priority and authority filters in SQL and an index on (authority, status, created_at); stats via GROUP BY; duplicate check limited to an indexed area+issue window of the last 7 days; IDs from a database sequence; SQLite replaced by PostgreSQL with PostGIS so real boundaries can replace typed areas; jurisdiction records stored in a table behind a cache.